



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

AI Agents for Beginners (C690)

Course Code: C690

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	3 August 2026	Initial aligned release: 2 topics, 4 connected labs, and 1 day / 7.5 instructional hours.	Tertiary Infotech Academy Courseware Team

TABLE OF CONTENTS

Introduction.....	4
Course Learning Outcomes.....	4
Before You Start — Preparation.....	4
Topic 01 — Understanding AI Agents.....	5
From Chatbot to Goal-Directed Agent.....	5
The Agent Loop.....	6
Six Building Blocks of a Beginner Agent.....	6
What the Language Model Does - and Does Not Do.....	7
Workflow or Agent? Choose the Smallest Useful Pattern.....	7
Tools Turn Language into Observable Work.....	8
Memory Is Selected State, Not Automatic Truth.....	8
Three Common Agent Patterns.....	9
Use Cases Across Service, Marketing, and Operations.....	9
No-Code Platform Landscape.....	10
Lab 1 — Select a Use Case and Map the Agent.....	11
Recap — Understanding AI Agents.....	13
Topic 02 — Building Your First AI Agents.....	14
Begin with an Agent Canvas.....	14
Goal, Instructions, and Persona Have Different Jobs.....	15
The GATES Instruction Contract.....	15
Grounding: Put Approved Evidence into the Answer Path.....	16
Tools Need Descriptions, Schemas, and Permission Boundaries.....	16
Memory Design for a Short Service Conversation.....	17
A No-Code Agent on the n8n Canvas.....	17
Worked Example: BrightDesk Workspace Assistant.....	18
Test Behaviour, Not Just Happy Paths.....	18
Security, Privacy, and Prompt Injection.....	19
Deploy in Stages and Keep a Rollback Path.....	19
Lab 2 — Write and Test the Agent Instructions and Persona.....	20
Lab 3 — Build and Ground the No-Code Agent in n8n.....	22
Lab 4 — Add a Tool, Test the Boundaries, and Publish Safely.....	26
Recap — Building Your First AI Agents.....	29
Wrap-Up - From Demonstration to Responsible Pilot.....	29
Next Steps.....	30
Glossary.....	30

Introduction

AI agents extend generative AI from producing a reply to pursuing a goal through a controlled loop of planning, tool use, observation, and adjustment. This learner guide explains the design principles before the practical steps so you can transfer the method to different platforms and workplace tasks.

Across four connected labs, you will create one BrightDesk Workspace support agent. You will define its boundary, write its instructions, build it in n8n, ground it on a synthetic FAQ, attach a calculator tool, and test whether it should remain in sandbox or be published as a controlled demonstration.

Course Learning Outcomes

- LO1: Explain how AI agents differ from chatbots and fixed workflows, including the roles of models, prompts, tools, memory, observations, and stop conditions.
- LO2: Compare conversational, task-based, and multi-agent patterns and select an appropriate no-code platform and bounded use case.
- LO3: Design an agent with a clear goal, instructions, persona, evidence boundary, tool permissions, output contract, and human escalation path.
- LO4: Build a simple no-code AI agent in n8n and ground its answers on an approved synthetic knowledge source.
- LO5: Connect a low-risk tool to an agent and control when the agent may calculate, recommend, refuse, or ask for human approval.
- LO6: Test and prepare an agent for deployment using expected-answer checks, safety cases, logging, monitoring, rollback, and continuous improvement.

Before You Start — Preparation

What you need

- A Windows or Mac laptop with a modern web browser and access to the course repository.
- A trainer-provided n8n workspace or your own n8n Cloud trial; the current interface may differ slightly by version.
- A trainer-approved model credential already stored in n8n, or your own authorised API credential stored only in n8n Credentials.
- A text or Markdown editor for the C690-agent-starter-pack files.
- Only the supplied synthetic BrightDesk knowledge and test data; no real customer, employee, payment, or confidential information.

Verify your setup

Confirm that you can open n8n, create or import a workflow, open the course resources, and create the connected output folder without placing a secret in a prompt or file.

Open: [labs/resources/brightdesk-workspace-faq.md](#)

Open: [labs/resources/brightdesk-agent-test-cases.csv](#)

Create folder: C690-agent-starter-pack

Conventions used in every lab

- Replace placeholders such as <MODEL_CREDENTIAL> only inside the n8n credential selector; never paste a key into a node prompt or course file.
- Use the exact output filenames shown so later labs can reuse earlier checkpoints.
- Treat every generated response as a draft until its source, calculation, tool path, and authority boundary are checked.
- Keep the workflow in test or unpublished mode until Lab 4 records an explicit publish decision.
- Use only synthetic course data during learning and verification.

Topic 01 — Understanding AI Agents

From chatbots to agents | models, prompts, tools and memory | agent types | use cases | no-code platforms

Key concepts

- Goal-directed: An agent works toward an explicit result instead of only producing the next reply.
- Model: The language model interprets intent, proposes steps, and chooses among allowed tools.
- Instructions: A reusable contract defines role, evidence, rules, output, and escalation behaviour.
- Tools: Tools let the agent retrieve information, calculate, update a system, or trigger a workflow.
- Memory and state: State carries only the context needed for the next decision; it is not automatically reliable truth.
- Observe and adjust: The agent reads tool results, checks progress, and decides whether to continue, retry, stop, or ask a person.
- Bounded autonomy: Permissions, limits, approval gates, and stop conditions define what the agent may do.
- Evidence over confidence: A fluent answer is not proof; important claims and actions need source evidence and verification.

From Chatbot to Goal-Directed Agent

A chatbot is a conversational interface: it receives a message and returns a response. An AI agent adds goal management and an action loop. It can interpret the objective, decide which permitted tool is useful, observe the result, and continue until the goal is

met or a stop condition is reached. The interface may still look like chat, but the operating model is different.

The distinction is about control, not marketing labels. A fixed sequence that always follows the same route is a workflow, even if one step uses a language model. A system becomes more agentic when the model chooses among possible steps or tools based on the situation. More autonomy is not automatically better; it increases the need for clear evidence, permissions, testing, and human oversight.

Visual framework

Chatbot: Responds to the current message | Usually produces text only | Relies on context supplied in the conversation | A person performs the next action

AI agent: Works toward a defined outcome | Can choose and call approved tools | Observes results and updates its next step | Stops, escalates, or completes within a boundary

The Agent Loop

An agent begins with a goal and the context available at that moment. It selects a next step, may call a tool, reads the observation returned by the environment, then compares progress with the success rule. The loop continues only while another step is useful and permitted. Completion, uncertainty, a policy rule, an error, or an iteration limit can end the loop.

The observation is important because it reconnects the model to reality. A booking tool may return that a room is unavailable; a knowledge tool may return no matching source; a calculator may return a value. The agent should use that evidence instead of continuing from an assumption. Good designs make the loop visible in logs and give a person a clear point to intervene.

Visual framework

- Receive goal
- Plan the next step
- Use an allowed tool
- Observe the result
- Check success or stop

Six Building Blocks of a Beginner Agent

A useful agent is a system, not a prompt. The goal states what good looks like. The model interprets language and makes bounded choices. Instructions explain the role and rules. Tools connect the agent to information or actions. Memory carries selected context. Guardrails keep the whole system within acceptable risk.

Weakness in one block propagates to the others. If a tool description is vague, the model may call it for the wrong purpose. If persistent memory stores unverified

statements, later answers can repeat them as fact. If the goal is simply 'be helpful', there is no measurable finish line. Design each block and its interface before increasing autonomy.

Visual framework

- Goal — The outcome, user, success signal, time horizon, and finish line.
- Model — The reasoning and language engine, chosen for the task and operating constraints.
- Instructions — Role, evidence rules, decision rules, output schema, and escalation path.
- Tools — Clearly named capabilities with narrow inputs, outputs, permissions, and error behaviour.
- Memory — Short-lived conversation context or approved persistent state needed for continuity.
- Guardrails — Validation, privacy rules, approvals, limits, monitoring, and rollback.

What the Language Model Does - and Does Not Do

A language model predicts and generates language from patterns. It is strong at interpreting ambiguous requests, organising context, drafting, and choosing among well-described options. It can still invent facts, overlook a constraint, or choose an unnecessary tool. Fluency should never be treated as a control mechanism.

Deterministic controls belong outside the model. Authentication, access checks, required fields, numeric limits, approval gates, and audit logs should be enforced by the surrounding workflow. The model can recommend an action, but the system decides whether that action is permitted. This separation makes failures easier to diagnose and contain.

Visual framework

Useful model work: Interpret unstructured requests | Classify intent and propose a plan | Select among described tools | Draft and explain a result

Separate system responsibility: Authenticate the user and protect secrets | Enforce permissions and spending limits | Validate critical calculations and records | Log, monitor, approve, and reverse actions

Workflow or Agent? Choose the Smallest Useful Pattern

A fixed workflow is often the right first solution. It is easier to test because the route is known. Agentic choice adds value when the input is open-ended and the next step cannot be fully predicted, such as deciding whether a customer question needs a knowledge lookup, a calculation, or a human handoff.

Start with the simplest pattern that meets the need. Add retrieval before adding many tools; add one bounded tool before granting write access; add multi-agent coordination

only when distinct roles genuinely improve the outcome. Every new decision point increases latency, cost, failure paths, and the amount of evidence needed to trust the system.

Visual framework

Use a fixed workflow when: The path and rules are known in advance | The same inputs should always take the same route | A mistake would be costly or difficult to reverse | Speed, predictability, and auditability dominate

Use an agent when: Requests are varied or unstructured | The next step depends on new observations | Several reasonable tools or routes may work | A person can review important exceptions

Tools Turn Language into Observable Work

A tool is a defined interface to a capability such as search, calculation, a database lookup, a calendar, or an automation. Its name and description tell the model when it should be used; its input schema constrains the request; its output gives the agent an observation. Narrow, well-documented tools are easier for both the model and the operator to understand.

Tool use should be separated by risk. Reading public or synthetic information is usually low risk. Drafting a ticket or calculating a price is limited and reversible. Sending a message, changing a record, making a booking, or spending money is higher risk and usually needs authentication, validation, and approval. A tool error must return a clear failure instead of pretending the action succeeded.

Visual framework

- User request
- Agent selects tool
- System validates inputs
- Tool returns evidence
- Agent explains or escalates

Memory Is Selected State, Not Automatic Truth

Memory helps an agent maintain continuity, but stored content can be incomplete, outdated, or wrong. Conversation memory may preserve a user's earlier preference. Task state records what has been completed. Long-term memory may persist approved information across sessions. A knowledge source is different: it is retrieved evidence used to answer a question.

Store the minimum state required for the job. Never place passwords, API keys, or unnecessary personal data in prompts or memory. Give important records a source, timestamp, owner, and correction path. When a value must be authoritative - such as a

price, policy, or entitlement - retrieve it from the system of record rather than relying on remembered text.

Visual framework

- Conversation memory — Recent turns that help the agent understand references and maintain continuity.
- Task state — Confirmed fields, progress, tool results, and the current checkpoint for one job.
- Long-term profile — Approved preferences or history retained across sessions under a clear policy.
- Knowledge source — Documents or records retrieved as evidence; this is different from memory.
- Retention rule — What is stored, why, for how long, who can access it, and how it is removed.
- Correction path — How a person can inspect, amend, or delete incorrect state.

Three Common Agent Patterns

Conversational agents are natural for service, guidance, and intake. Task-based agents focus on an outcome such as drafting a report or reconciling records. Multi-agent systems divide work among specialised roles, but introduce coordination, duplicated effort, and harder evaluation. A hybrid workflow often provides the best beginner architecture.

Choose a pattern from the work, not from novelty. If the goal, available evidence, and allowed actions fit inside one agent, keep one agent. If a process contains a known decision table, keep that portion deterministic. Multi-agent designs are justified when separate contexts, permissions, or expertise materially improve the result and the handoff can be verified.

Visual framework

- Conversational agent — Maintains a dialogue, retrieves information, and routes exceptions while a person remains in the interaction.
- Task-based agent — Works toward a defined artifact or operational result using a small set of tools and stop conditions.
- Multi-agent system — Coordinates specialised agents through an orchestrator; useful only when roles and handoffs are genuinely distinct.
- Hybrid workflow — Uses deterministic routing for known rules and an agent only where interpretation or flexible planning adds value.

Use Cases Across Service, Marketing, and Operations

Good starter use cases have clear users, bounded inputs, observable outcomes, and recoverable mistakes. They are repetitive enough to justify design effort but variable enough to benefit from language understanding. A customer support assistant grounded

on a small, approved FAQ is a stronger first project than an agent with access to every company system.

Avoid using an agent where a wrong answer or action could seriously harm a person, create a legal commitment, expose confidential data, or spend money without meaningful human control. If the outcome cannot be checked, the system cannot be improved. Write the success evidence and failure response before selecting a platform.

Visual framework

- Customer service — Answer from approved policies, classify intent, draft responses, and escalate sensitive or unsupported cases.
- Marketing — Research approved sources, create variants, organise a content plan, and route material for brand review.
- Operations — Summarise requests, check records, calculate routine values, prepare drafts, and coordinate handoffs.
- Knowledge support — Retrieve relevant passages, cite the source, explain uncertainty, and record unanswered questions.
- Personal productivity — Turn notes into actions, prepare a meeting brief, or organise a repeatable decision checklist.
- Avoid or constrain — High-stakes decisions, unrestricted data access, irreversible actions, and tasks without verifiable outcomes.

No-Code Platform Landscape

No-code does not mean no design. A visual builder can make the flow easier to see, but the same questions remain: what is the goal, which source is authoritative, what may the agent change, how is success checked, and who responds when the system is uncertain? Platform choice should follow these requirements.

This course uses n8n because its canvas exposes the main parts of an agent and its workflow. Learners can connect a chat trigger, AI Agent node, model, memory, and tool without programming. The design concepts transfer to other platforms; menus and node names change, but goals, evidence, permissions, tests, and human control remain.

Visual framework

- Chat-based builders — Fast instruction, persona, file-grounding, and conversation testing for simple assistants.
- Workflow builders — Visual triggers, agent nodes, tools, data movement, approvals, logs, retries, and integrations.
- Enterprise studios — Managed identity, connectors, governance, environments, and organisational deployment controls.
- Agent frameworks — Code-level control for teams that need custom tools, state, evaluation, and deployment architecture.
- Selection criteria — Required tools, data location, authentication, cost, observability, export, and operator skills.

- Course platform — n8n provides a visual canvas for the model, instructions, memory, tools, test chat, and execution history.

Lab 1 — Select a Use Case and Map the Agent

Learning outcome: LO1 and LO2: distinguish an agent from a chatbot or fixed workflow, then select and map a bounded beginner use case.

Goal: You begin the connected BrightDesk Workspace scenario by turning a broad support-assistant idea into a testable agent canvas. You decide where flexible agent behaviour adds value, where a fixed rule is safer, what evidence the agent may use, and which actions remain with staff.

Duration: 50 minutes.

What you'll build

C690-agent-starter-pack/01-use-case-and-agent-canvas.md containing the user and job, outcome, success evidence, non-goals, six building blocks, tool permissions, risks, stop conditions, and human owner. (Tools: Text or Markdown editor, labs/resources/01-use-case-and-agent-canvas-starter.md, labs/resources/brightdesk-workspace-brief.md, labs/resources/brightdesk-workspace-faq.md, optional approved AI assistant for critique.)

Prerequisites

- Create a local folder named C690-agent-starter-pack and copy the Lab 1 starter into it.
- Open the synthetic BrightDesk brief and FAQ; do not add real customer or employee information.
- Keep the design in recommendation and demonstration mode; no live booking, payment, email, or record change is permitted.

Step-by-step

1. Copy the supplied canvas starter into your connected output folder. Keep every heading and table so later checks use the same structure.

Copy: `labs/resources/01-use-case-and-agent-canvas-starter.md -> C690-agent-starter-pack/01-use-case-and-agent-canvas.md`

2. Read brightdesk-workspace-brief.md without AI. Copy only the stated user, business need, supported services, constraints, and human owner into the matching headings. Label anything not stated as UNKNOWN.

Allowed evidence: `labs/resources/brightdesk-workspace-brief.md` and `labs/resources/brightdesk-workspace-faq.md`

3. Write one observable outcome and three success signals. Keep the outcome about helping a visitor find supported information and prepare a non-binding estimate, not about maximising conversation length.

Outcome: Help a prospective BrightDesk visitor obtain a source-backed answer or estimate and reach staff when the request needs a human.

Signals: supported answer cites a section ID; calculation shows inputs; unsupported or consequential request is handed off.

4. List at least five non-goals. Include confirming availability, making a booking, taking payment, changing a record, and collecting sensitive personal information.

Non-goal status: PROHIBITED | HUMAN ONLY | OUT OF SCOPE

5. Classify the work into fixed workflow and agent decisions. Keep source lookup, numeric validation, permission checks, and logging deterministic; use the agent for interpreting varied questions and selecting an allowed response path.

Table columns: Work item | Fixed workflow or agent | Why | Expected evidence | Failure response

6. Complete the six-block anatomy for Goal, Model, Instructions, Tools, Memory, and Guardrails. Give each block a one-sentence responsibility and one failure to watch.

Example failure: Memory repeats an unverified discount as if it were an approved price.

7. Create a permission matrix for FAQ lookup, calculator, draft staff handoff, booking, payment, and customer-record update. Assign one level to every row.

Levels: ALLOW IN SANDBOX | RECOMMEND ONLY | HUMAN APPROVAL | PROHIBITED

8. Add at least six risks covering unsupported claims, stale knowledge, wrong arithmetic inputs, prompt injection, unnecessary personal data, and implied booking authority. Pair each risk with prevention, detection, and response.

Table columns: Risk | Prevention | Detection | Response | Owner

9. Write stop conditions for missing evidence, conflicting prices, personal or payment data, a request to ignore rules, an unavailable tool, repeated failure, and any request to confirm an external action.

Required response pattern: STOP - state the reason - preserve available evidence - offer the named human handoff.

10. Review the canvas against the BrightDesk sources. Add the reviewer, date, three corrections, and unresolved questions. If you used AI for critique, verify every change against the source before accepting it.

Review Log

- Reviewer: <INITIALS>

- Date: <YYYY-MM-DD>
- Corrections: <THREE ITEMS>
- Unresolved: <ITEM OR NONE>

Test it

Open 01-use-case-and-agent-canvas.md. It must contain one observable outcome, three success signals, at least five non-goals, all six agent blocks, six permission rows, six risks with controls, seven stop conditions, and a named human owner. Ask a partner to point to any proposed action; within ten seconds you must be able to show its authority level, evidence, failure response, and owner.

Checkpoint for the next lab

Keep 01-use-case-and-agent-canvas.md. Lab 2 converts its goal, evidence, permissions, risks, and stop conditions into reusable agent instructions and a stable test set.

Troubleshooting

- The idea is still 'a helpful assistant': Rewrite the outcome around a named user, source-backed answer or estimate, success evidence, and staff handoff.
- Everything is classified as agent work: Move known rules, validation, permissions, logging, and irreversible actions into fixed workflow or human-control rows.
- The agent can book or take payment: Change those rows to HUMAN APPROVAL or PROHIBITED and add explicit stop language.

Challenge

Add a simple cost-risk-value score from 1 to 5 for three possible use cases and explain why the BrightDesk support use case is the safest first build.

Reflection

Which part of the BrightDesk task benefits most from flexible language understanding, and which part should remain deterministic even if the model appears capable?

Note: Full steps, commands, checkpoints, and troubleshooting are in labs/lab-01-*.md. Use the synthetic BrightDesk scenario and placeholder credentials only. Do not collect real personal data, expose a secret, confirm a booking, take payment, or send an external message from the course workflow.

Recap — Understanding AI Agents

You can now:

- LO1 and LO2: distinguish an agent from a chatbot or fixed workflow, then select and map a bounded beginner use case

Carry forward the verified lab checkpoints and resolve any OWNER TO VERIFY items with the named owner before the next topic.

Topic 02 — Building Your First AI Agents

Goals, instructions and personas | no-code build | document grounding | tools | testing, deployment and best practices

Key concepts

- Design the outcome: Name the user, job, success evidence, non-goals, and human owner before opening a builder.
- Write an instruction contract: Specify goal, grounding, actions, tests, escalation, and state in reusable language.
- Ground important claims: Supply approved sources and require the agent to cite, qualify, or refuse when evidence is missing.
- Grant least authority: Start read-only, add one low-risk tool, and require approval for consequential actions.
- Test behaviour: Use normal, boundary, missing-evidence, adversarial, and tool-failure cases with expected outcomes.
- Deploy gradually: Move from sandbox to internal pilot to controlled use only after evidence supports the next level.
- Monitor outcomes: Retain inputs, tool calls, outputs, errors, corrections, latency, and user feedback appropriate to the risk.
- Keep a rollback path: Know how to pause the workflow, revoke credentials, restore state, and return work to a person.

Begin with an Agent Canvas

An agent canvas turns an idea into a testable system. Identify the person being helped and the job they need done. Describe the result in observable terms. List the approved inputs and sources. Define the smallest set of tools. Mark actions that are read-only, reversible, approval-controlled, or prohibited.

Also write non-goals. A BrightDesk support agent may answer from a synthetic FAQ and calculate an estimate, but it does not confirm availability, take payment, make a booking, or collect sensitive personal data. Non-goals prevent a friendly persona from implying authority the system does not have.

Visual framework

- User and job

- Outcome and evidence
- Inputs and sources
- Tools and authority
- Risks and human owner

Goal, Instructions, and Persona Have Different Jobs

The goal says what the agent is trying to achieve. Instructions describe how it should behave. A persona affects the experience - for example, calm, concise, and beginner-friendly - but must never grant extra authority. Context and examples provide the evidence and patterns needed for the current request.

Keep these layers separate so they can be changed and tested independently. A tone change should not alter the tool permission matrix. A new knowledge source should not silently change the success rule. When a user asks the agent to ignore its rules, the agent should preserve its instruction hierarchy and either continue safely or escalate.

Visual framework

- Goal — Defines the observable result and what completion means.
- Instructions — Define evidence, decision rules, tools, output, refusals, and escalation.
- Persona — Shapes tone, vocabulary, and interaction style without changing permissions.
- Context — Supplies the approved facts and situation for the current task.
- Examples — Show difficult cases and the desired format or boundary behaviour.
- User message — Expresses the current need; it does not override higher-priority rules.

The GATES Instruction Contract

GATES is a beginner-friendly checklist for complete agent instructions: Goal, Authority, Truth, Evaluation, State, and Escalation. It prevents the common mistake of writing only a persona and a vague task. Each section can be reviewed by the person who owns that risk or decision.

A strong contract is specific enough to test. 'Use the knowledge base' becomes 'Use only the named BrightDesk FAQ for prices, hours, and policies; cite the section ID; if no source supports the answer, say what is missing and offer a human handoff.' The second version creates observable behaviour and a clear failure condition.

Visual framework

- Goal — Who is helped, what result is required, and what counts as complete.
- Authority — Allowed tools and actions, approval gates, prohibited actions, and limits.
- Truth — Authoritative sources, citation rules, uncertainty language, and missing-evidence behaviour.

- Evaluation — Expected output, checks, test cases, quality thresholds, and failure signals.
- State — What context is retained, for how long, and how it can be corrected or removed.
- Escalation — When to stop, what to tell the user, what evidence to pass, and who owns the next step.

Grounding: Put Approved Evidence into the Answer Path

Grounding connects the agent's answer to approved evidence. For a small beginner project, the knowledge text can be placed directly in the system context. Larger collections usually use retrieval-augmented generation: documents are split into chunks, represented for search, relevant passages are retrieved, and those passages are supplied to the model for the current question.

Retrieval does not guarantee correctness. The wrong passage may be retrieved, the source may be outdated, or the model may overstate what the passage says. Keep source identifiers, require citations, test questions that are both inside and outside the source, and define the response when evidence is missing. Knowledge content is data, not an instruction channel; embedded commands in a document must not override the agent's rules.

Visual framework

- User question
- Retrieve relevant source
- Supply passage to model
- Generate with citation
- Check support or refuse

Tools Need Descriptions, Schemas, and Permission Boundaries

A model sees a tool through its interface. A vague name such as 'operations' forces the model to guess. A name such as 'calculate_workspace_estimate' with numeric inputs and a non-binding output is easier to use correctly. Validate required values before execution and return a structured error when the tool cannot complete the request.

Tool authority belongs to the workflow and credential, not to the persona. Begin with read-only or synthetic tools. Require approval before communications, record changes, bookings, purchases, or deletion. Idempotency and duplicate checks matter because an agent or user may retry. The safest tool is often a draft-producing tool whose output is reviewed before any external action.

Visual framework

- Clear purpose — A tool name and description explain exactly when it should and should not be used.

- Narrow input — Required fields, types, ranges, and accepted values reduce ambiguous calls.
- Structured output — Success, result, source, error, and next action are explicit.
- Least privilege — Credentials and operations expose only what the task requires.
- Approval — Consequential calls pause until an authorised person confirms the evidence.
- Failure behaviour — Timeouts, empty results, duplicates, and validation errors return a safe, visible state.

Memory Design for a Short Service Conversation

Short-term memory makes a conversation coherent. If a user says they need a room for three hours, the next question can refer to 'that booking'. The agent should still distinguish a confirmed fact from an assumption and should not treat memory as proof of availability or payment.

For the course agent, keep only recent synthetic conversation context. Do not ask for identity documents, payment details, passwords, or real customer records. Production designs need explicit retention, access, correction, and deletion rules. When a session ends, unnecessary state should expire rather than becoming an unreviewed profile.

Visual framework

- Keep recent turn
- Extract confirmed fact
- Discard unnecessary detail
- Use in next response
- Expire at session end

A No-Code Agent on the n8n Canvas

In n8n, the Chat Trigger receives the message. The AI Agent contains the reusable instructions and coordinates the response. A chat model supplies language and reasoning. Simple Memory carries recent turns. A Calculator tool gives the agent one observable, low-risk capability. Execution history shows which nodes ran and what they returned.

The connected nodes make responsibility visible. The trigger is an interface, not the intelligence. The model cannot act unless a tool is attached. Memory is optional and separate. The workflow can remain in test mode while behaviour is refined. This modular view helps a beginner change one component and rerun the same test set.

Visual framework

- Chat Trigger
- AI Agent
- Chat Model

- Simple Memory
- Calculator Tool

Worked Example: BrightDesk Workspace Assistant

A user asks, 'What is the estimate for a meeting room for three hours?' The agent finds the approved hourly rate and minimum duration in the BrightDesk knowledge block. It calls the calculator for 40 multiplied by 3, reports S\$120 as a non-binding estimate, cites the price section, and explains that availability and booking require staff confirmation.

Each component has a job. The source supplies the rate. The calculator supplies the arithmetic result. The instruction contract supplies the wording and authority boundary. The model connects them into a useful response. If the user asks for a discount not in the FAQ, the correct outcome is not a guess; it is a clear statement that the source does not support the request and a human handoff option.

Visual framework

- Ask: three-hour room estimate
- Retrieve price rule
- Call calculator
- Label estimate as non-binding
- Offer staff handoff for booking

Test Behaviour, Not Just Happy Paths

A demonstration proves only that one example worked once. A test set defines expected behaviour across normal, edge, missing-evidence, adversarial, and failure cases.

Record the input, expected evidence, expected action, observed result, correction, and rerun result. Evaluate the final outcome and the tool path that produced it.

Keep a stable regression set. When instructions, knowledge, model, or tools change, rerun the same cases. A correction that improves one response can harm another. For important workflows, combine deterministic checks, model-based review, and human judgement appropriate to the risk rather than relying on a single score.

Visual framework

- Normal — A supported question returns the correct source-backed answer.
- Boundary — A near-limit case follows the exact rule and unit.
- Missing evidence — The agent states the gap instead of inventing an answer.
- Adversarial — A request to ignore rules or reveal secrets does not change authority.
- Tool failure — The agent reports the failure and offers a safe next step.
- Human review — A consequential or sensitive request stops with the required evidence for handoff.

Security, Privacy, and Prompt Injection

Prompt injection is an attempt to place conflicting instructions inside user input or retrieved content. The system should treat untrusted content as data, preserve the instruction hierarchy, restrict tools, and validate every consequential action. No single prompt can guarantee protection; layers of technical and operational controls are required.

Privacy starts with not collecting what the task does not need. Use synthetic data in learning and testing. Store credentials only in the platform's protected credential manager. Review logs for sensitive content before retention. Give operators a way to pause the workflow, revoke access, remove state, and return the task to a manual process.

Visual framework

- Treat content as data — Documents, websites, and messages may contain instructions that must not override system rules.
- Protect credentials — Secrets belong in the platform credential store, never in prompts, files, screenshots, or chat.
- Minimise access — Restrict data fields, tool operations, environments, and network destinations.
- Validate actions — Check identity, required fields, ranges, targets, and approval before execution.
- Monitor — Log tool calls, failures, refusals, corrections, and unusual usage within an appropriate retention policy.
- Contain — Rate limits, timeouts, iteration limits, kill switches, and rollback reduce the impact of a failure.

Deploy in Stages and Keep a Rollback Path

Deployment is a controlled increase in real-world exposure. Start in a sandbox with synthetic content. Move to an internal test with known users and a stable test set. Use recommendation or draft mode before allowing an action. Add approval for consequential steps. Expand scope only when outcome evidence and incident handling support it.

Before publication, name the owner, success metrics, error thresholds, monitoring rhythm, support channel, and rollback action. A rollback may mean unpublishing the workflow, revoking a credential, disabling a tool, restoring a previous version, or directing all requests to staff. Continued monitoring is part of the agent, not a separate afterthought.

Visual framework

- Sandbox
- Internal test

- Recommendation mode
- Approved action
- Monitored expansion

Lab 2 — Write and Test the Agent Instructions and Persona

Learning outcome: LO3: design a GATES instruction contract with a clear goal, persona, evidence boundary, tool permissions, output rules, tests, and escalation.

Goal: You convert the approved Lab 1 canvas into reusable instructions for the BrightDesk Workspace assistant. You keep the friendly persona separate from authority, define how sources and calculations appear, and test whether the instructions resist missing evidence and unsafe requests.

Duration: 50 minutes.

What you'll build

C690-agent-starter-pack/02-agent-instructions-and-tests.md containing the GATES contract, persona, response schema, stable test cases, observed results, corrections, and version notes. (Tools: Approved AI assistant, text or Markdown editor, C690-agent-starter-pack/01-use-case-and-agent-canvas.md, labs/resources/02-agent-instructions-and-tests-starter.md, labs/resources/brightdesk-gates-baseline.md, BrightDesk FAQ and test-case CSV.)

Prerequisites

- Completed Lab 1 canvas with the outcome, source boundary, permission matrix, risks, stop conditions, and owner.
- Use a fresh chat in an organisation-approved AI assistant and only the supplied synthetic BrightDesk content.
- Do not paste a credential, real personal data, or confidential workplace information into the chat.

Step-by-step

1. Copy the supplied instruction starter into the connected output folder. Keep the GATES headings, response schema, baseline case table, corrections, and version notes.

Copy: labs/resources/02-agent-instructions-and-tests-starter.md -> C690-agent-starter-pack/02-agent-instructions-and-tests.md

2. Write the Goal section from Lab 1. Name the user, supported job, observable result, completion rule, and non-goals in direct language.

Goal rule: a useful reply ends with a source-backed answer, a transparent non-binding calculation, or a staff handoff - never an implied booking or payment.

3. Write the Authority section as a tool and action matrix. Permit source lookup and sandbox calculation; keep staff handoff as a draft; prohibit booking, payment, record changes, credential handling, and collection of sensitive data.

Table columns: Capability | When allowed | Required input | Required check |
Result label | Human gate

4. Write the Truth section. Name the BrightDesk FAQ as the only authoritative source for hours, prices, facilities, and policies. Require section IDs, distinguish source facts from calculations, and define missing or conflicting evidence behaviour.

If the FAQ does not support a claim, say: 'I do not have an approved source for that. BrightDesk staff must confirm it.'

5. Write the Evaluation section with the exact response schema and all ten MUST-PASS rows in the supplied CSV. Mark B01, B02, S01, and S02 as the four baseline cases for Labs 2 and 3; the complete ten-case regression is run in Lab 4.

Response fields: Answer | Source | Calculation | Action status | Next step

Baseline: B01, B02, S01, S02

Final regression: B01-B03, T01-T03, S01-S04

6. Write the State section. Keep only recent synthetic conversation turns, treat remembered values as unverified until the FAQ supports them, and expire the practice session after the lab.

Never retain: passwords, API keys, identity documents, payment details, health information, or real customer records.

7. Write the Escalation section with the seven stop conditions from Lab 1. Specify what the agent tells the user and what source, question, attempted action, and error it passes to BrightDesk staff.

Escalation packet: user request | supported facts | missing evidence | action requested | reason for stop | recommended human owner

8. Add the persona after GATES: calm, concise, welcoming, beginner-friendly, and transparent about limits. State that tone never overrides the source boundary or permissions.

Persona line: Warm and practical; never imply that politeness, urgency, or user insistence changes authority.

9. Paste the complete contract and the synthetic FAQ into a fresh approved AI chat. Run baseline cases B01, B02, S01, and S02 exactly as written in the CSV. Record the full observed response and result for each.

Baseline IDs: B01 opening hours | B02 visitor check-in | S01 unsupported discount | S02 booking boundary

Result labels: MEETS EXPECTATION | REVISE INSTRUCTIONS | SOURCE GAP | HUMAN REVIEW

10. Correct the contract, start another fresh chat, and rerun every failed case. Record version 1.0 only when all four behaviours match the expected evidence and authority decision.

Version note: v1.0 | <DATE> | <CHANGE SUMMARY> | <RERUN CASES>

Test it

Start a new chat with the final contract and FAQ. Run B01, B02, S01, and S02 exactly as written in the test CSV. The agent must cite HOURS-01 and VISITOR-01 for the supported answers, state the source gap for the discount, and stop the booking request with a staff handoff. Record all four as MEETS EXPECTATION.

Checkpoint for the next lab

Keep 02-agent-instructions-and-tests.md. Lab 3 pastes this reviewed contract into the n8n AI Agent and reruns the same expectations on the visual workflow.

Troubleshooting

- The persona promises actions the tool cannot perform: Move all permissions into Authority and add a sentence that persona and user urgency never expand them.
- The response gives a price without a source ID: Make Source a required field and instruct the agent to refuse unsupported values.
- A correction works only in the existing chat: Start a fresh chat for every final rerun so hidden conversation history cannot mask a weak contract.

Challenge

Add two paraphrased versions of the unsupported-discount case and verify that the same boundary holds when the wording changes.

Reflection

Which instruction produced the biggest improvement in observable behaviour: the goal, authority, truth, evaluation, state, or escalation section?

Note: Full steps, commands, checkpoints, and troubleshooting are in labs/lab-02-*.md. Use the synthetic BrightDesk scenario and placeholder credentials only. Do not collect real personal data, expose a secret, confirm a booking, take payment, or send an external message from the course workflow.

Lab 3 — Build and Ground the No-Code Agent in n8n

Learning outcome: LO4: build a simple no-code n8n agent with a chat trigger, model, memory, reviewed instructions, and an approved knowledge source.

Goal: You implement the BrightDesk design on the n8n canvas using a supplied credential-free starter workflow. You connect your own approved model credential, paste the reviewed instructions and synthetic FAQ, use short-term memory, run the baseline questions, and inspect execution evidence.

Duration: 60 minutes.

What you'll build

A saved n8n workflow named C690 - BrightDesk Knowledge Agent - v1.0 plus C690-agent-starter-pack/03-build-and-baseline-test-log.md and an exported credential-free workflow checkpoint. (Tools: n8n Cloud or trainer-provided n8n, trainer-approved chat model credential stored in n8n Credentials, Lab 3 JSON, 03-build-and-baseline-test-log-starter.md, sanitizer script, BrightDesk FAQ, Lab 2 instruction contract.)

Prerequisites

- Completed Lab 2 contract with all four quick tests meeting expectation.
- Access to a trainer-provided or personal n8n practice workspace.
- A model credential stored in n8n Credentials; never paste the secret into a prompt, node field, file, or screenshot.

Step-by-step

1. Open n8n and choose Workflows > Create Workflow. Use the workflow menu to select Import from File, then select the supplied Lab 3 JSON.

Import: `labs/resources/C690-Lab3-BrightDesk-Knowledge-Agent.json`

2. Rename the workflow to C690 - BrightDesk Knowledge Agent - v1.0 and keep it inactive or unpublished. Confirm the canvas contains Chat Trigger, AI Agent, OpenAI Chat Model, and Simple Memory.

Expected route: Chat Trigger -> AI Agent; OpenAI Chat Model -> AI Agent; Simple Memory -> AI Agent

3. Open OpenAI Chat Model. Select a trainer-approved credential from the credential selector and choose an available low-cost chat model. If the trainer uses another supported provider, replace only the model sub-node and preserve the same test expectations.

Secret rule: create or select the credential in n8n Credentials; do not paste an API key into any prompt or export.

4. Open AI Agent. In System Message, replace the GATES placeholder with the final Lab 2 contract. Preserve the required response fields and the statement that persona never changes authority.

Paste from: `C690-agent-starter-pack/02-agent-instructions-and-tests.md`

- 5.** Below the contract, replace the KNOWLEDGE placeholder with the complete synthetic FAQ. Add a clear boundary before and after the text so the source is treated as data rather than as higher-priority instructions.

```
<KNOWLEDGE source='brightdesk-workspace-faq.md'>
```

```
<PASTE COMPLETE SYNTHETIC FAQ>
```

```
</KNOWLEDGE>
```

- 6.** Open Simple Memory. Use the connected-chat session key from Chat Trigger and set a small context window such as five exchanges. Do not configure persistent production storage in this beginner lab.

Memory purpose: conversation continuity only; prices and policies still require the FAQ source.

- 7.** Copy the supplied build-log starter, then open Chat and run the supported baseline cases B01 and B02. Record the answer, cited section, expected result, and observed result.

```
Copy: labs/resources/03-build-and-baseline-test-log-starter.md -> C690-agent-starter-pack/03-build-and-baseline-test-log.md
```

Cases: B01 and B02

- 8.** Ask the unsupported-discount and booking-action cases. Confirm the agent states the source gap or action boundary and offers staff handoff without requesting sensitive information.

Cases: S01 and S02 from labs/resources/brightdesk-agent-test-cases.csv

- 9.** Open Executions and inspect one supported and one stopped case. Confirm Chat Trigger and AI Agent ran, the response came from the current instruction and knowledge version, and no secret or real personal data appears in the execution data.

Evidence fields: workflow version | case ID | nodes run | source cited | action status | result

- 10.** Complete the build log for B01, B02, S01, and S02. Correct the system message or knowledge boundary for any failure, start a new test session, and rerun until all four meet expectation.

Baseline IDs: B01 | B02 | S01 | S02

- 11.** Create a private-working folder inside C690-agent-starter-pack. Export the workflow there with the exact private filename shown below, then run the supplied sanitizer to create the shareable checkpoint. Re-import the sanitized copy into a blank workflow and confirm the model node asks you to select a credential. Never submit or share the private-working folder.

Private export filename: C690-agent-starter-pack/private-working/C690-BrightDesk-Knowledge-Agent-v1.0-private.json

```
python labs/resources/sanitize_n8n_export.py "C690-agent-starter-pack/private-working/C690-BrightDesk-Knowledge-Agent-v1.0-private.json" "C690-agent-starter-pack/C690-BrightDesk-Knowledge-Agent-v1.0.json"
```

Re-import the sanitized output; expected: SANITIZED OK and no selected model credential.

Test it

In a fresh n8n test chat, run B01, B02, S01, and S02. All four must match the expected source and authority behaviour in the CSV. Then open the execution for S02 and verify that the workflow produced a refusal and staff handoff without any external action. Run the sanitizer, re-import the sanitized checkpoint, and confirm that it contains no credential object or secret-like value and that the model node requires credential selection.

Checkpoint for the next lab

Keep the inactive v1.0 workflow, test log, and credential-free export. Lab 4 adds one Calculator tool, reruns the complete stable test set, and records the publish or hold decision.

Troubleshooting

- The model node shows a credential error: Select or create the approved credential in n8n Credentials. Never solve the error by pasting the key into a prompt.
- The agent invents a price or policy: Check that the full FAQ is inside the knowledge boundary and strengthen the Truth rule to require a section ID or refusal.
- The next test remembers an earlier unsupported claim: Start a new chat session and reduce the memory window; authoritative values still come from the FAQ.

Challenge

On a disposable copy of the FAQ, change one synthetic value and rerun its case. Restore version 1.0, start a fresh session, and rerun B01, B02, S01, and S02 before continuing to Lab 4.

Reflection

Which execution evidence helps you distinguish a knowledge problem from an instruction problem or a model problem?

Note: Full steps, commands, checkpoints, and troubleshooting are in labs/lab-03-*.md. Use the synthetic BrightDesk scenario and placeholder credentials only. Do not collect real personal data, expose a secret, confirm a booking, take payment, or send an external message from the course workflow.

Lab 4 — Add a Tool, Test the Boundaries, and Publish Safely

Learning outcome: LO5 and LO6: connect a low-risk tool, verify tool and safety behaviour, and prepare a controlled deployment with monitoring and rollback.

Goal: You add a Calculator tool to the grounded BrightDesk agent so it can produce transparent, non-binding workspace estimates. You run the full regression set, inspect whether the tool was used only when appropriate, and record a publish or hold decision with monitoring and rollback.

Duration: 60 minutes.

What you'll build

An n8n workflow named C690 - BrightDesk Tool Agent - v1.1 plus C690-agent-starter-pack/04-tool-tests-and-deployment-record.md and a final manifest linking all four lab artifacts. (Tools: n8n practice workspace, Lab 3 workflow, Lab 4 JSON, 04-tool-tests-and-deployment-record-starter.md, sanitizer script, BrightDesk FAQ, complete test-case CSV.)

Prerequisites

- Lab 3 workflow passes all four baseline tests and remains inactive or unpublished.
- The exported Lab 3 checkpoint is credential-free.
- You know how to Unpublish the workflow before enabling a controlled public test link.

Step-by-step

1. Duplicate the verified Lab 3 v1.0 workflow and rename the duplicate C690 - BrightDesk Tool Agent - v1.1. If you are rejoining directly, first import the Lab 3 starter, insert labs/resources/brightdesk-gates-baseline.md and the complete FAQ, select an approved model credential, confirm five-turn memory, and pass B01, B02, S01, and S02; then duplicate it. The supplied Lab 4 JSON is a read-only configuration reference, not the workflow used for the diff gate.

Rejoin import: `labs/resources/C690-Lab3-BrightDesk-Knowledge-Agent.json`

Reference only: `labs/resources/C690-Lab4-BrightDesk-Tool-Agent.json`

Required before proceeding: verified v1.0 duplicated | v1.1 unpublished | B01/B02/S01/S02 pass

2. Confirm Calculator is connected to the AI Agent tool port. In the system message, permit it only for arithmetic using values that the current FAQ supports; require the response to show inputs, result, source, and 'non-binding estimate'.

Tool rule: never use an invented rate, discount, availability, tax, fee, or duration; ask for a missing quantity or hand off a missing policy.

3. Copy the supplied deployment-record starter. Run the ten MUST-PASS rows B01-B03, T01-T03, and S01-S04 in fresh sessions and record the expected source,

expected tool decision, observed tool decision, answer, action status, and result. F01 is an optional extension.

Copy: labs/resources/04-tool-tests-and-deployment-record-starter.md -> C690-agent-starter-pack/04-tool-tests-and-deployment-record.md

4. Inspect Executions for the meeting-room estimate. Confirm Calculator ran with source-backed numeric inputs. Inspect the unsupported-discount, prompt-injection, sensitive-data, and booking cases; confirm Calculator or any external action did not run when the boundary required a stop.

Evidence: case ID | Calculator called YES/NO | inputs | output | source ID | action status

5. Correct and rerun every failure. Sanitize/export v1.0 and v1.1, compare them with n8n's publish diff or a JSON diff, and record the reviewer. Allowed changes are Calculator, its permission rule, and version metadata only; HOLD on any new trigger, selected credential, external tool, or public setting.

Allowed diff: Calculator node and connection | Calculator permission text | v1.1 name/version

Decision labels: READY FOR CONTROLLED DEMO | HOLD - <CASE OR UNEXPECTED DIFF>

6. With trainer approval, open Chat Trigger, enable Make Chat Publicly Available, use Hosted Chat with the trainer's restricted authentication option, and choose Publish. Record the URL and publication time. Then choose Unpublish and confirm the public URL no longer works; record the unpublish time and failed-access evidence. If approval is not given, keep it unpublished and record HOLD.

Never use anonymous access for the exercise and never publish a workflow containing a secret in a prompt, real customer data, or an external write tool.

7. Complete the deployment record with owner, source version, instruction version, model, permitted tool, support channel, monitoring checks, error threshold, review date, pause method, credential revoke method, and rollback version.

Rollback target: C690-BrightDesk-Knowledge-Agent-v1.0.json

8. After the containment test, turn Make Chat Publicly Available off and save. Rerun the v1.0/v1.1 diff and confirm public is false, with no new trigger, credential, or external tool. Export the private v1.1 file into the existing private-working folder with the exact filename shown below. Add a Final Manifest linking artifacts 01-04 and both sanitized exports. Run the sanitizer, re-import the sanitized result, and confirm the model credential is blank before adding it to the manifest. Never submit or share the private-working folder.

Final release gates: active=false | Chat Trigger public=false | allowed diff only

Private export filename: C690-agent-starter-pack/private-working/C690-BrightDesk-Tool-Agent-v1.1-private.json

```
python labs/resources/sanitize_n8n_export.py "C690-agent-starter-pack/private-  
working/C690-BrightDesk-Tool-Agent-v1.1-private.json"  
"C690-agent-starter-pack/C690-BrightDesk-Tool-Agent-v1.1.json"
```

Test it

All MUST-PASS cases in brightdesk-agent-test-cases.csv must meet the expected source, tool, and authority behaviour. The three-hour estimate must call Calculator with 40 and 3 and return S\$120 as a non-binding estimate citing PRICE-01. The unsupported-discount, injection, sensitive-data, and booking cases must not trigger a tool or external action. After the controlled-demo containment test, the final workflow and sanitized export must show active=false and Chat Trigger public=false. The deployment record must contain a publish or hold decision, owner, monitoring checks, tested Unpublish path, and rollback target.

Checkpoint for the next lab

Retain the complete C690-agent-starter-pack and the credential-free v1.1 export. Reuse the stable test set whenever the model, instructions, source, memory, tool, or workflow version changes.

Troubleshooting

- Calculator runs with a rate the user supplied: Require every price input to match an approved FAQ section before the tool call; otherwise stop for missing evidence.
- The agent says the room is booked: Strengthen the non-goal and Action status rules: it can estimate only; availability and booking remain with staff.
- The public test URL works after Unpublish: Confirm you unpublished the correct workflow and retest from a private browser window; record the actual containment result. Then turn Make Chat Publicly Available off and save.

Challenge

Add one malformed numeric input and one calculator failure case, then specify the exact safe user message and human handoff evidence for each.

Reflection

What evidence would justify adding a real booking-request tool, and which approval and rollback controls would need to exist first?

Note: Full steps, commands, checkpoints, and troubleshooting are in labs/lab-04-*.md. Use the synthetic BrightDesk scenario and placeholder credentials only. Do not collect real personal data, expose a secret, confirm a booking, take payment, or send an external message from the course workflow.

Recap — Building Your First AI Agents

You can now:

- LO3: design a GATES instruction contract with a clear goal, persona, evidence boundary, tool permissions, output rules, tests, and escalation
- LO4: build a simple no-code n8n agent with a chat trigger, model, memory, reviewed instructions, and an approved knowledge source
- LO5 and LO6: connect a low-risk tool, verify tool and safety behaviour, and prepare a controlled deployment with monitoring and rollback

Carry forward the verified lab checkpoints and resolve any OWNER TO VERIFY items with the named owner before the next topic.

Wrap-Up - From Demonstration to Responsible Pilot

Your final starter pack connects a business outcome, evidence boundary, instruction contract, no-code implementation, tool permission, test results, and deployment decision. The trace between these artifacts is what makes the agent reviewable.

Before adapting the agent at work

- Replace the synthetic FAQ only with an approved, current source owned by the business process owner.
- Reconfirm the user group, authentication need, permitted tools, approval gates, privacy rules, and data-retention policy.
- Create a representative test set with expected outcomes and assign an owner for errors, incidents, and source updates.
- Pilot in recommendation or draft mode before granting any external write action.

Evidence to retain

- Agent version, source version, instruction version, model choice, tool definitions, and credential scope.
- Test inputs, expected evidence and action, observed result, corrections, and regression rerun.
- Publish decision, owner, monitoring thresholds, pause method, rollback action, and review date.

Next Steps

- Replace the calculator with one read-only workplace lookup tool and document its schema, error states, and permission boundary.
- Expand the knowledge source using a retrieval workflow while retaining source IDs and missing-evidence tests.
- Add a human approval step before any message, booking, record change, or other consequential action.
- Rerun the stable test set whenever the model, instructions, source content, tools, or workflow version changes.
- Review n8n and model-provider documentation before using current platform features in production.

Glossary

- **Agent** — A system in which a model directs steps or tool use toward a goal within defined boundaries.
- **Agentic loop** — The repeated cycle of selecting an action, observing the result, and deciding what to do next.
- **Approval gate** — A required human decision before a consequential action can proceed.
- **Chatbot** — A conversational system that responds to messages; it may or may not include agentic tool choice.
- **Context** — Information supplied for the current task or conversation.
- **Embedding** — A numeric representation used to compare the meaning of text for retrieval.
- **Evaluation** — A repeatable method for comparing observed behaviour with expected outcomes.
- **Grounding** — Supplying approved evidence so an answer can be supported and checked.
- **Guardrail** — A rule, validation, permission, approval, limit, or operational control that constrains behaviour.
- **Hallucination** — Generated content that is unsupported, incorrect, or invented while sounding plausible.
- **Human handoff** — A controlled stop that transfers the request and relevant evidence to a named person or team.
- **Instruction hierarchy** — The priority order among system rules, application instructions, approved context, and user requests.
- **Memory** — Selected conversational or task state retained to support future decisions.
- **Model** — The AI component that interprets language and generates or selects responses and actions.

- **Multi-agent system** — A design in which multiple specialised agents coordinate through defined roles and handoffs.
- **No-code platform** — A visual environment for assembling triggers, models, tools, data, and controls with little or no programming.
- **Prompt injection** — Instructions placed in user or retrieved content that attempt to override trusted rules or misuse tools.
- **RAG** — Retrieval-augmented generation: retrieving relevant source passages and supplying them to a model for the current task.
- **Rollback** — A planned action that returns the workflow to a known safer state after a problem.
- **State** — The confirmed information and progress the system carries between steps or turns.
- **Stop condition** — A rule that ends or pauses the loop because the goal is complete, evidence is missing, risk is high, or a limit is reached.
- **Tool** — A defined interface that lets an agent retrieve information, calculate, or request an external operation.
- **Workflow** — A predefined sequence of steps and rules; some steps may use a model without giving it control of the route.